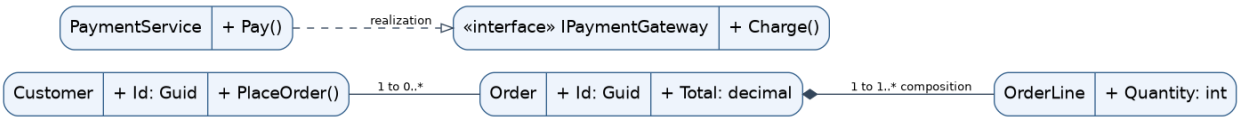
Senior .NET Interview Cheat Sheet

UML diagrams • C4 • SOLID • C# • Interview questions and answers

Visual examples are simplified teaching diagrams. In interviews, name the notation and explain failure cases, ownership, and trade-offs.

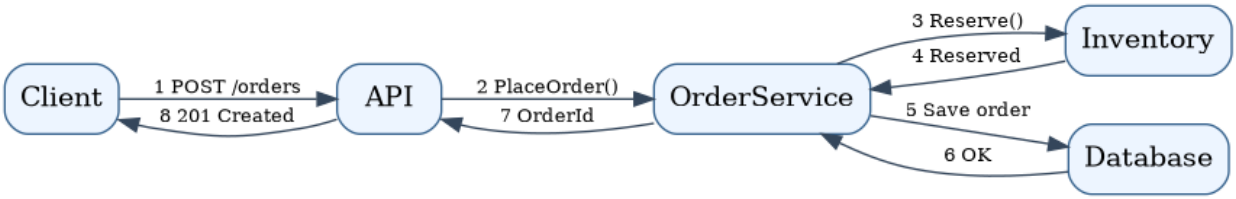
Class Diagram - relationships



Q: What is the difference between association, aggregation and composition?

Association is a general relationship. Aggregation is a weak whole-part relationship where the part may outlive the whole. Composition expresses strong ownership and dependent lifetime. Here an Order owns its OrderLines. The dashed hollow-triangle arrow shows interface realization.

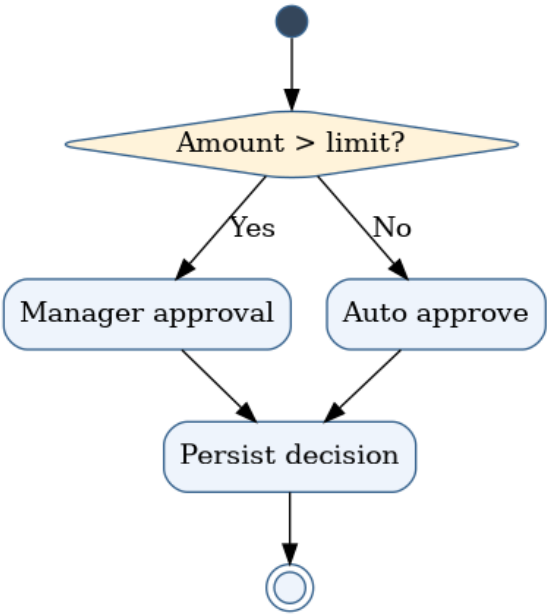
Sequence Diagram - checkout



Q: When do you use a sequence diagram?

To show runtime interactions and message order. During a system-design interview explain the happy path, then discuss timeout, retry, idempotency, transaction boundaries and compensation when inventory reservation succeeds but persistence fails.

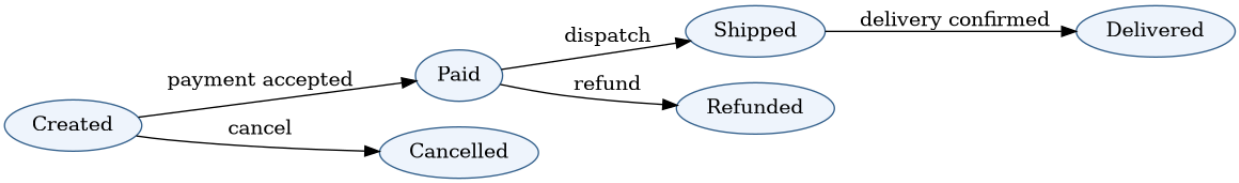
Activity Diagram - approval workflow



Q: How is an activity diagram different from a sequence diagram?

Activity diagrams emphasize control flow, branches and parallel work. Sequence diagrams emphasize which component calls which other component, and in what order.

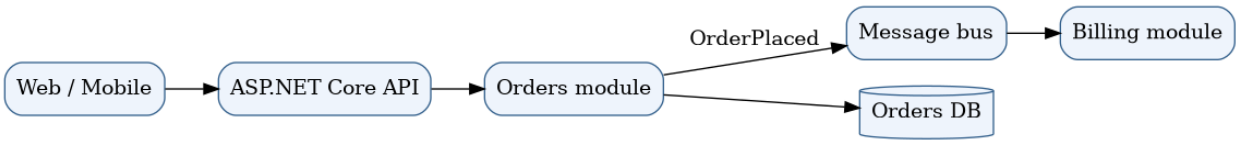
State Machine - order lifecycle



Q: Why model states explicitly?

An explicit state machine prevents invalid transitions, such as shipping a cancelled order. Enforce transitions in the domain model and consider concurrency control to prevent competing updates.

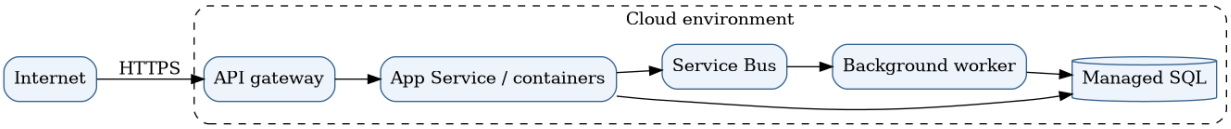
Component Diagram - modular backend



Q: What does a component diagram communicate?

Logical components, their responsibilities and dependencies. It does not necessarily tell you where processes run; a deployment diagram handles runtime nodes and infrastructure.

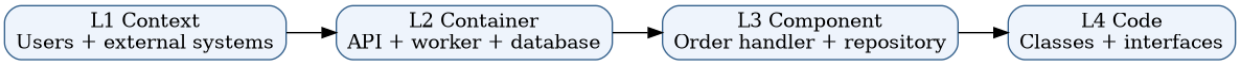
Deployment Diagram - cloud hosting



Q: What should a senior engineer add to a deployment diagram?

Availability zones, autoscaling, network boundaries, secrets, monitoring, failure domains and data recovery assumptions. Explain the operational trade-offs, not just boxes.

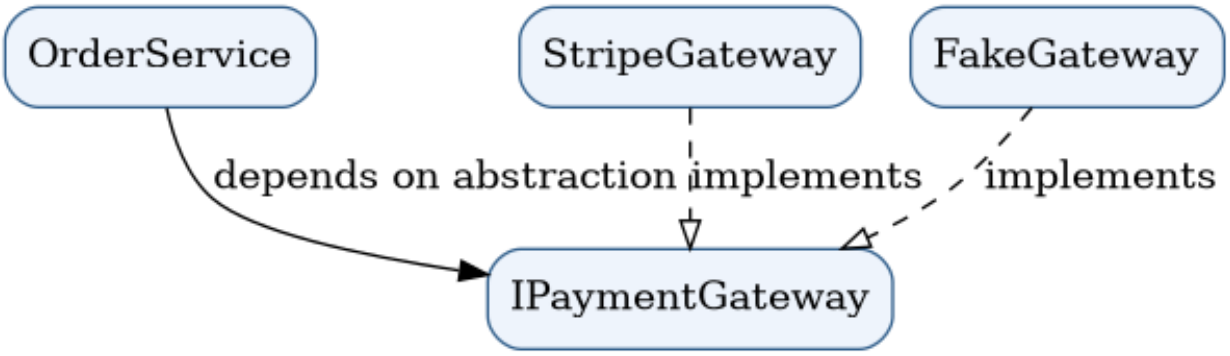
C4 Model - Context to Code



Q: How do C4 and UML work together?

C4 gives progressive architectural zoom levels. UML can describe details within a C4 level, for example a sequence diagram for a request or a class diagram for domain relationships. C4 Container means an executable or data store, not necessarily a Docker container.

SOLID - Dependency Inversion example



Q: What is the difference between DIP and DI?

DIP is a design principle: high-level policy depends on abstractions, not concrete infrastructure. DI is a technique to supply dependencies, often through constructors. A DI container is optional; dependency injection alone does not guarantee good abstraction boundaries.

SOLID - five principles with interview answers

S - Single Responsibility — One reason to change, not necessarily one method. Separate pricing policy from email notification and persistence.

O - Open/Closed — Extend behavior via strategies or polymorphism instead of repeatedly modifying a large switch statement.

L - Liskov Substitution — Subtypes must preserve the contract. Do not strengthen preconditions or weaken postconditions.

I - Interface Segregation — Prefer focused interfaces (IReadRepository, IWriteRepository) when consumers need different capabilities.

D - Dependency Inversion — Domain/application policies reference abstractions; adapters implement them. Keep abstractions at the appropriate boundary.

C# example - constructor injection and DIP

```
public interface IPaymentGateway
{
    Task ChargeAsync(decimal amount, CancellationToken ct);
}
public sealed class CheckoutService(IPaymentGateway gateway)
{
    public Task CheckoutAsync(decimal total, CancellationToken ct)
        => gateway.ChargeAsync(total, ct);
}
// Composition root:
// services.AddScoped<IPaymentGateway, StripePaymentGateway>();
```

Bonus interview questions

Q: When is a class diagram overkill?
A: When the key issue is service interaction, deployment or workflow; use a sequence, deployment or activity diagram instead.

Q: What should a diagram leave out?
A: Details that do not support the decision. Show boundaries, responsibilities and critical dependencies; keep notation consistent.

Q: How would you model distributed failure?
A: Start with sequence diagram: mark timeout, retries, duplicate messages, outbox and idempotency. Then add deployment failure domains.

Q: Can SOLID lead to overengineering?
A: Yes. Apply abstractions where volatility, testing or substitution justify them. Avoid one-interface-per-class without a clear boundary.